

中山大學
SUN YAT-SEN UNIVERSITY

数据库系统原理实验 | 实验十：

基于 Django 与 SQL Server 的图书管理系统开发

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

课 程 教 师: 赖韩江

起 始 日 期: 2025 年 12 月 1 日

结 束 日 期: 2025 年 12 月 5 日

实 验 地 点: B202

目录

1 实验概述 **2**

2 实验报告 **2**

2.1 技术选型与理由 2
2.2 环境搭建与准备 2
2.3 框架总览与使用介绍 3
2.4 代码设计与实现 4
2.4.1 数据库连接测试 4
2.4.2 Django 数据库配置 5
2.4.3 模型定义 5
2.4.4 表单定义 6
2.4.5 视图逻辑 7
2.4.6 URL 配置 8
2.4.7 模板文件 8
2.5 实验结果展示 10
2.6 实验总结 13

1 实验概述

使用 JaveServlet/JSP/Django/Flask 或其它 Web 应用框架，构建一个 web 界面程序，实现以用户界面的形式对图书借阅数据库系统 JY 的图书表的增删改查功能。报告要求附上代码文件、设计过程以及功能测试。

2 实验报告

2.1 技术选型与理由

本项目采用 Django 框架作为后端，结合 SQL Server 数据库，利用 mssql-django 和 pyodbc 实现高效的数据交互。Django 提供完善的 ORM、表单和模板机制，适合快速开发和维护。SQL Server 作为企业级数据库，性能和安全性有保障。

2.2 环境搭建与准备

- 安装 Python 3.8+，Django、mssql-django、pyodbc 依赖。
- 安装 ODBC Driver 17 for SQL Server。
- 创建虚拟环境并激活，安装 requirements.txt。

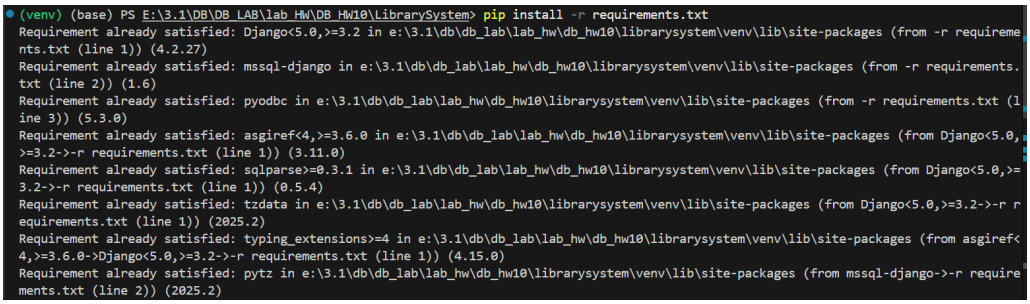


图 1: 执行 pip install -r requirements.txt 的结果

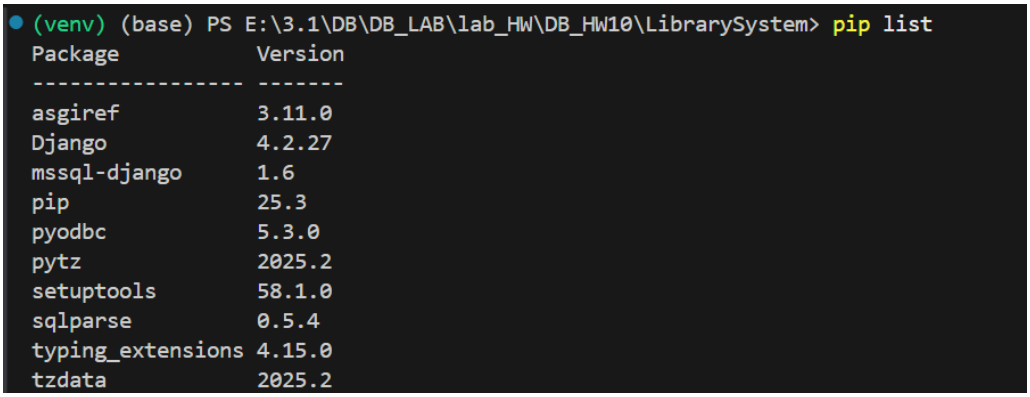


图 2: pip list 查看依赖安装情况

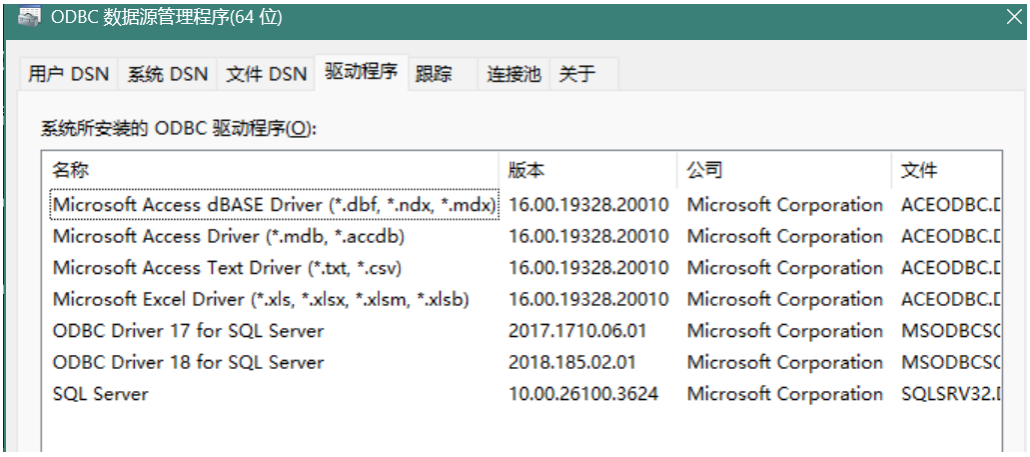


图 3: ODBC Driver 17 安装成功

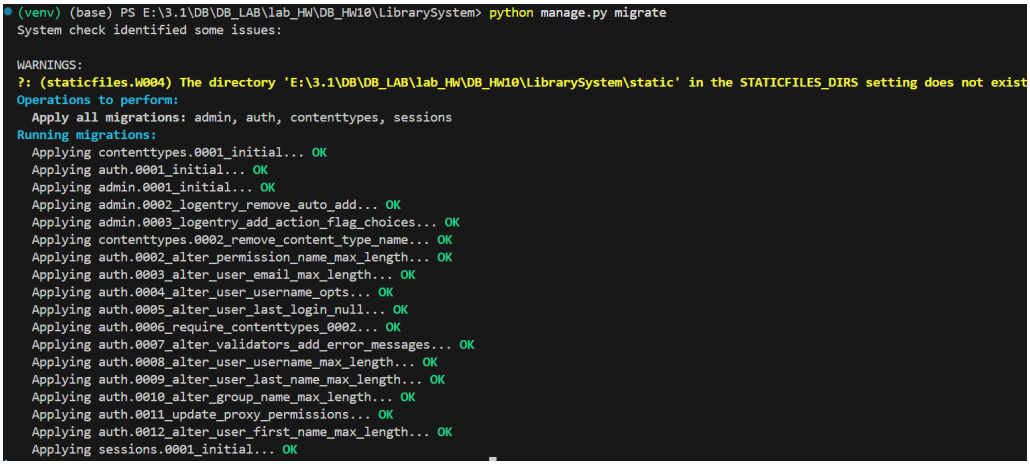


图 4: python manage.py migrate 初始化数据库

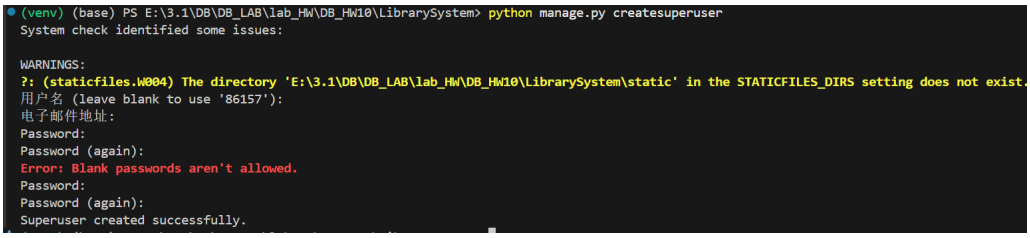


图 5: python manage.py createsuperuser 创建超级用户

2.3 框架总览与使用介绍

项目主要目录结构如下，并对主要文件功能进行简要注释：

```
LibrarySystem/
  books/
    admin.py          # Django后台管理配置，注册Book模型
    apps.py           # 应用配置文件
```

```
  forms.py          # 图书表单定义，简化表单校验与渲染
  models.py         # ORM模型定义，映射数据库book表
  urls.py           # books应用的路由配置
  views.py          # 视图函数，处理业务逻辑和数据交互
  ...
jy_project/
  settings.py       # 全局配置，包括数据库、应用注册等
  urls.py           # 项目级URL路由入口
  ...
templates/
  books/
    base.html               # 基础模板，统一页面布局
    index.html              # 首页模板，展示图书总量
    book_list.html          # 图书列表与搜索页面
    book_form.html          # 图书新增/编辑表单页面
    book_confirm_delete.html # 删除确认页面
    ...
test_conn.py       # pyodbc数据库连接测试脚本
requirements.txt   # Python依赖包清单
...
```

使用方法：

1. 进入项目根目录，激活虚拟环境（如 `venv\Scripts\activate`）。
2. 安装依赖：`pip install -r requirements.txt`
3. 数据库连接测试（可选）：`python test_conn.py`
4. 初始化 Django 内部表（如首次部署）：`python manage.py migrate`
5. 创建超级用户（如需后台管理）：`python manage.py createsuperuser`
6. 启动开发服务器：`python manage.py runserver`
7. 浏览器访问 `http://127.0.0.1:8000/` 进入首页，或 `/books/` 进入图书管理界面。

2.4 代码设计与实现

本项目采用 Django 的 MTV（模型-模板-视图）架构，结合 SQL Server 现有表结构，前后端分离，所有业务逻辑均通过视图函数和表单完成，模板只负责渲染和交互。下面结合主要代码，说明各部分的设计与实现思路。

2.4.1 数据库连接测试

Listing 1: test_conn.py

```
import pyodbc
```

```
conn_str = (
    "DRIVER={ODBC Driver 17 for SQL Server};"
    "SERVER=FORTUNE,1433;"
    "DATABASE=JY;"
    "Trusted_Connection=yes;"
)
print("尝试连接: ", conn_str)
try:
    conn = pyodbc.connect(conn_str, timeout=5)
    cur = conn.cursor()
    cur.execute("SELECT TOP 3 book_id, book_name, book_price FROM book")
    rows = cur.fetchall()
    print("查询结果样例（最多3条）: ")
    for r in rows:
        print(r)
    conn.close()
    print("连接与查询成功")
except Exception as e:
    print("连接或查询失败, 错误: ", e)
```

设计思路说明：在正式开发前，先用 pyodbc 独立脚本测试数据库连通性和权限，确保环境无误再进行 Django 项目开发。这样可以快速定位驱动、网络、权限等底层问题，避免后续遇到“连不上数据库”而难以排查。

2.4.2 Django 数据库配置

Listing 2: jy_project/settings.py

```
DATABASES = {
    'default': {
        'ENGINE': 'mssql',
        'NAME': 'JY',
        'HOST': 'FORTUNE',
        'PORT': '1433',
        'OPTIONS': {
            'driver': 'ODBC Driver 17 for SQL Server',
            'trusted_connection': 'yes',
        },
    }
}
```

设计思路说明：采用 mssql-django 作为 Django 的数据库后端，配置与 pyodbc 一致，保证开发、测试、生产环境一致性。使用 Windows 身份验证，提升安全性，减少明文密码泄露风险。

2.4.3 模型定义

Listing 3: books/models.py

```
from django.db import models

class Book(models.Model):
```

```
    book_id = models.CharField(max_length=8, primary_key=True, verbose_name="图书编
        号")
    book_name = models.CharField(max_length=50, verbose_name="图书名称")
    book_isbn = models.CharField(max_length=17, verbose_name="ISBN")
    book_author = models.CharField(max_length=10, verbose_name="作者")
    book_publisher = models.CharField(max_length=50, verbose_name="出版社")
    book_price = models.DecimalField(max_digits=19, decimal_places=4, verbose_name=
        "价格")
    interviews_times = models.IntegerField(verbose_name="借阅次数", default=0)

    class Meta:
        managed = False
        db_table = 'book'
        verbose_name = '图书'
        verbose_name_plural = '图书管理'

    def __str__(self):
        return self.book_name
```

设计思路说明：模型字段严格映射数据库表结构，类型、长度、主键与数据库一致。通过 managed=False，Django 只做数据操作，不会自动迁移或修改表结构，适合对接已有数据库。这样既能用 Django ORM 的便利，又不会破坏原有数据表。

2.4.4 表单定义

Listing 4: books/forms.py

```
from django import forms
from .models import Book

class BookForm(forms.ModelForm):
    class Meta:
        model = Book
        fields = ['book_id', 'book_name', 'book_isbn', 'book_author', '
            book_publisher', 'book_price', 'interviews_times']
        widgets = {
            'book_id': forms.TextInput(attrs={'class': 'form-control', 'placeholder
                ': '最多8位字符'}),
            'book_name': forms.TextInput(attrs={'class': 'form-control'}),
            'book_isbn': forms.TextInput(attrs={'class': 'form-control'}),
            'book_author': forms.TextInput(attrs={'class': 'form-control'}),
            'book_publisher': forms.TextInput(attrs={'class': 'form-control'}),
            'book_price': forms.NumberInput(attrs={'class': 'form-control', 'step':
                 '0.01'}),
            'interviews_times': forms.NumberInput(attrs={'class': 'form-control', '
                value': 0}),
        }
```

设计思路说明：利用 Django ModelForm 自动生成表单，减少重复代码，保证表单校验与模型一致。通过 widgets 属性自定义前端样式，提升用户体验。这样既保证了数据安全性，也方便后续维护和扩展。

2.4.5 视图逻辑

Listing 5: books/views.py

```
from django.shortcuts import render, redirect, get_object_or_404
from .models import Book
from .forms import BookForm
from django.db.models import Q

def index(request):
    book_count = Book.objects.count()
    return render(request, 'books/index.html', {'book_count': book_count})

def book_list(request):
    query = request.GET.get('q', '').strip()
    books = Book.objects.all().order_by('book_id')
    if query:
        books = books.filter(
            Q(book_id__icontains=query) |
            Q(book_name__icontains=query) |
            Q(book_author__icontains=query) |
            Q(book_publisher__icontains=query) |
            Q(book_isbn__icontains=query)
        )
    return render(request, 'books/book_list.html', {'books': books, 'q': query})

def book_create(request):
    if request.method == 'POST':
        form = BookForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('book_list')
    else:
        form = BookForm()
    return render(request, 'books/book_form.html', {'form': form, 'title': '添加图
        书'})

def book_update(request, pk):
    book = get_object_or_404(Book, pk=pk)
    if request.method == 'POST':
        form = BookForm(request.POST, instance=book)
        if form.is_valid():
            form.save()
            return redirect('book_list')
    else:
        form = BookForm(instance=book)
    return render(request, 'books/book_form.html', {'form': form, 'title': '修改图
        书'})

def book_delete(request, pk):
```

```
    book = get_object_or_404(Book, pk=pk)
    if request.method == 'POST':
        book.delete()
        return redirect('book_list')
    return render(request, 'books/book_confirm_delete.html', {'book': book})
```

设计思路说明：每个视图函数对应一个功能页面，采用函数式视图，逻辑清晰。列表页支持多字段模糊查询，提升检索效率。增删改查均采用 Django 的表单机制，保证数据校验和安全。删除操作采用二次确认，防止误删。

2.4.6 URL 配置

Listing 6: jy_project/urls.py

```
from django.contrib import admin
from django.urls import path
from books import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.index, name='index'),
    path('books/', views.book_list, name='book_list'),
    path('books/add/', views.book_create, name='book_create'),
    path('books/edit/<str:pk>/', views.book_update, name='book_update'),
    path('books/delete/<str:pk>/', views.book_delete, name='book_delete'),
]
```

设计思路说明： URL 路由采用 RESTful 风格，便于理解和扩展。每个功能有独立入口，方便后续增加更多模块或 API。

2.4.7 模板文件

Listing 7: templates/books/book_list.html

```
{% extends 'books/base.html' %}
{% block content %}
<div style="display: flex; justify-content: space-between; align-items: center;">
    <h2>图书信息管理</h2>
    <form method="get" style="display: flex; align-items: center; gap: 10px;">
        <input type="text" name="q" value="{{ q|default:'' }}" placeholder="书名/作
            者/ISBN关键词" class="form-control" style="width: 200px;">
        <button type="submit" class="btn btn-primary">搜索</button>
    </form>
    <a href="{% url 'book_create' %}" class="btn btn-primary">+ 添加新书</a>
</div>
<table border="1" cellspacing="0" cellpadding="10" style="width: 100%; margin-top:
    20px; border-collapse: collapse;">
    <thead>
        <tr style="background-color: #f2f2f2;">
            <th>编号</th>
            <th>书名</th>
```

```
            <th>作者</th>
            <th>出版社</th>
            <th>ISBN</th>
            <th>价格</th>
            <th>借阅次数</th>
            <th>操作</th>
        </tr>
    </thead>
    <tbody>
        {% for book in books %}
        <tr>
            <td>{{ book.book_id }}</td>
            <td>{{ book.book_name }}</td>
            <td>{{ book.book_author }}</td>
            <td>{{ book.book_publisher }}</td>
            <td>{{ book.book_isbn }}</td>
            <td>{{ book.book_price|floatformat:2 }}</td>
            <td>{{ book.interviews_times }}</td>
            <td>
                <a href="{% url 'book_update' book.book_id %}" class="btn btn-edit"
                    >修改</a>
                <a href="{% url 'book_delete' book.book_id %}" class="btn btn-
                    danger">删除</a>
            </td>
        </tr>
        {% empty %}
        <tr>
            <td colspan="8" style="text-align: center;">暂无图书数据</td>
        </tr>
        {% endfor %}
    </tbody>
</table>
{% endblock %}
```

设计思路说明：模板采用继承机制，统一风格。列表页集成搜索、操作按钮，所有数据由后端渲染，保证安全和一致性。表单、确认等页面均复用基础模板，便于维护。

2.5 实验结果展示

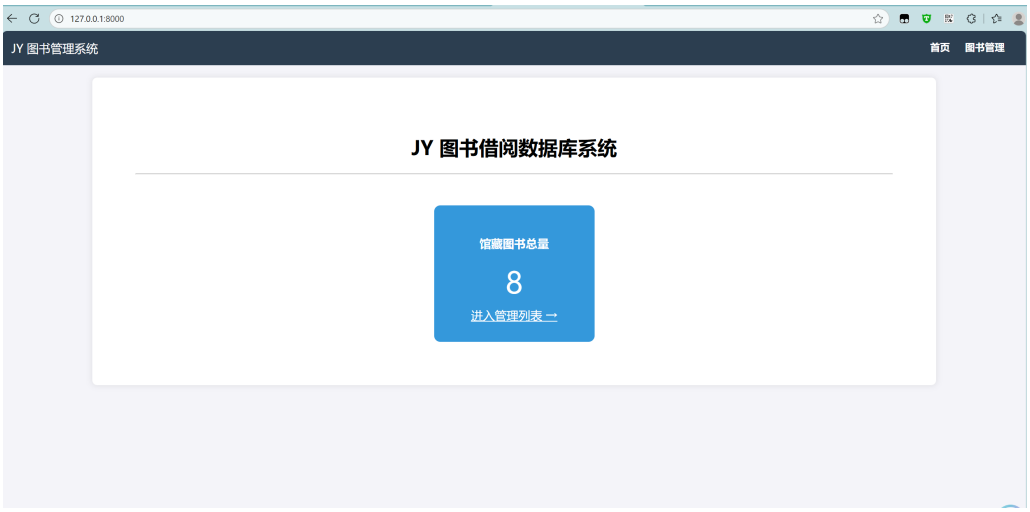


图 6: 系统首页截图



图 7: 图书信息管理界面截图

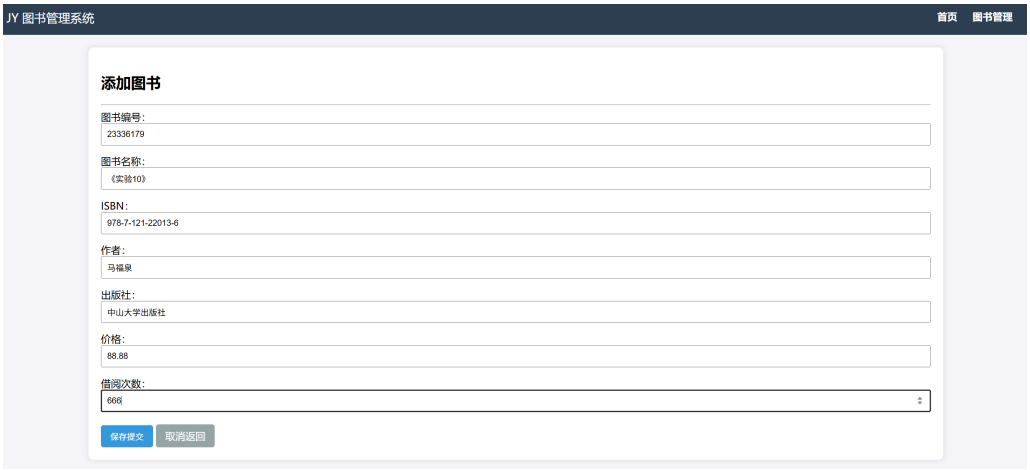


图 8: 添加新图书操作



图 9: 成功添加新图书的结果

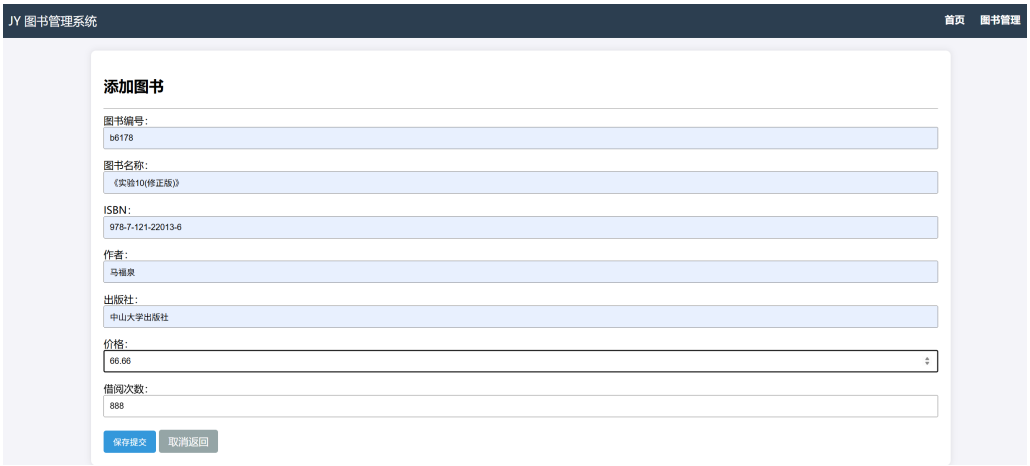


图 10: 修改图书过程



图 11: 修改图书结果



图 12: 按作者名查找图书



图 13: 按出版社查找图书

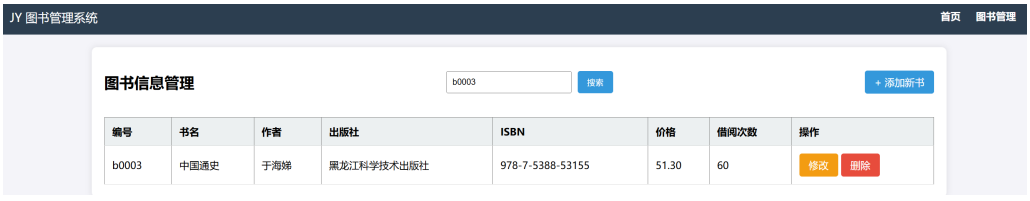


图 14: 按图书编号查找图书

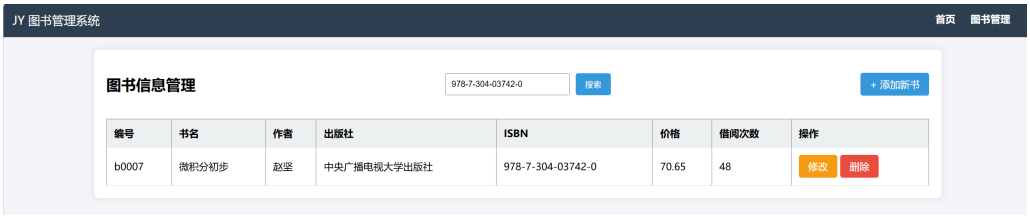


图 15: 按 ISBN 号查找图书



图 16: 按图书名查找图书

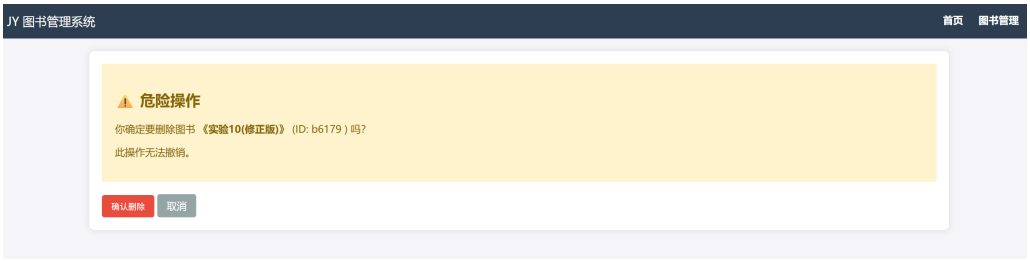


图 17: 执行删除图书操作的结果

2.6 实验总结

1. 实验收获

- 熟练掌握了 Django 与 SQL Server 的集成开发流程，初步理解了企业级数据库与现代 Web 框架的结合方式。
- 通过 ModelForm 和模板机制，提升了表单开发效率，保证了前后端数据一致性。
- 设计并实现多字段模糊查询，提升了系统的易用性和用户体验。
- 和前面实验相比，更进一步掌握了如何用 pyodbc 进行底层数据库连接测试，快速定位连接和权限问题。

2. 遇到的困难与解决

- **数据库连接失败**：初次连接 SQL Server 时遇到“Login failed”错误，排查发现是 Windows 身份验证未生效，后通过命令行以域用户身份运行并确认 trusted_connection 配置，成功解决。
- **ODBC 驱动兼容性问题**：未安装 ODBC Driver 17，导致 pyodbc 报错，后统一安装官方驱动并重启系统解决。
- **中文乱码问题**：查询结果在控制台输出时出现乱码，后检查数据库排序规则并在 Django 模板中使用 UTF-8 编码，页面显示正常。

- **权限不足**：测试新增和删除功能时，部分用户无写权限，联系管理员分配数据库权限后解决。
- **模板渲染错误**：模板变量拼写错误导致页面空白，通过 Django 调试工具定位并修正。